

Communication Flows & Privacy

Wallflower / moqplay · working reference · revised 15 August 2026 · companion to trust-model (who-can-see-what) and security-architecture (content encryption).

This traces the **wire-level messages** between the app, the two browser roles (broadcaster / watcher), the backend (broker + fleet), and the **chat** channel — with a privacy annotation on every hop, so we can see exactly **how a user could be identified, targeted, or revealed**, and where they're protected. Written to expose leaks, not to reassure.

Headline: the tokens and the media path are clean, but **precise geolocation (lat/lon) is captured server-side and persisted in D1 for both broadcasters and viewers**, and one endpoint (`GET /api/stats/stream/:id/viewers`) **re-exposes every viewer's stored geo publicly**. That is the single biggest deanonymization surface today (§6).

1. Global facts (apply to every flow)

- **OAuth is disabled → no account identity.** `getAuthenticatedUser()` returns a hardcoded anon user (`id:1`) — `index.ts:1459` , `auth.ts:25` . Every `user_id` written is `1` .
- **Every browser→Worker call is a Cloudflare request**, so Cloudflare/the Worker always sees the caller's **IP** and `request.cf` (lat/lon/city/region/country/colo/**asn**/asOrganization) — even when the body carries no identity.
- **Tokens are clean.** `MoqClaims = {put, get:[<broadcastName>], exp, cluster?}` (`moq-token.ts:43`). **No IP, geo, email, or account id in any token** — only the pseudonymous stream id + expiry.

2. The four channels

1 · Control

browser ↔ Worker
(`/api/*`). Carries IP + precise geo; some persisted to D1.

2 · Broker

Worker ↔ broker. Server-to-server; forwarded precise geo, no viewer IP.

3 · Chat

browser ↔ per-stream Durable Object (WebSocket). Plaintext; separate from media & relay.

4 · Media

browser ↔ relay box (WebTransport). Box sees browser IP; media is ciphertext (fleet flow).

Resolved 2026-08-15 — there is no longer a fifth channel. A *Discovery* plane used to sit here: browser ↔ public pkarr relays, exposing the browser IP and the nodeId being published or looked up to third-party infrastructure outside Cloudflare. The DHT path was removed from Wallflower entirely. No browser contacts a pkarr relay, and nothing about any broadcast is published to a public network. The two ⚠ findings in Group E below are closed.

3. Flow inventory (with privacy annotations)

Group A — Broadcaster browser ↔ Worker

Flow	Endpoint	Payload	Privacy
Go-live	POST /api/stats/broadcast (auth.ts:81)	{stream_id, publisher_cdn } + ?geo=	IP (cf). Precise geo captured server-side and stored in D1 broadcast_events (index.ts:1344,1364). Returns pub JWT + content key.
End	POST .../broadcast/:id/end (auth.ts:168)	none	IP (cf). No geo/token.
Settings	GET/POST /api/streams* (auth.ts:200,217,235)	stream id + flags	IP (cf). Pseudonymous stream id; user_id=1 .

Group B — Watcher browser ↔ Worker

Flow	Endpoint	Privacy
Route (fleet)	GET /api/streams/:id/route (auth.ts:132)	IP (cf). Reads ASN + AS-org (index.ts:682); brokered mode forwards precise viewer geo to broker . Returns viewer/pull JWT + content key.
Watch start	POST /api/stats/watch (auth.ts:176)	IP (cf). Precise viewer geo captured server-side and stored in D1 watch_events (index.ts:1428).
Watch end	POST /api/stats/watch/:id/end	IP (cf).
Live stats	GET /api/stats/live (auth.ts:280)	Re-exposes stored precise geo of all broadcasts+viewers.
Per-stream viewers	GET /api/stats/stream/:id/viewers (auth.ts:290)	⚠ PUBLIC. Anyone with a stream id reads every viewer's stored precise geo (index.ts:1257).

Group C — Worker ↔ broker / fleet

Flow	Endpoint	Privacy
Brokered assign	POST <broker>/cdn/assign (index.ts:1124)	No viewer IP (Worker-origin), but deliberately re-injects precise viewer geo {lat, lon, country, colo} (index.ts:1024). Auth = CDN_API_TOKEN (tenant, not user).
Direct assign/release	GET <box>/assign /release (index.ts:1073)	Server-to-server; no geo/IP forwarded in direct mode. Provision bearer (tenant).
Origin EID lookup	GET <box>/iroh?port= (index.ts:990)	Server-to-server; no user data.

Group D — Browser ↔ relay (WebTransport) — box sees the real browser IP

Flow	Connect URL	Privacy
Publisher	<code>https://<relay>/?jwt=</code> (main.ts:1247)	Box sees broadcaster IP. Token = stream id scope.
Viewer	<code>https://<relay>/?jwt=</code> (main.ts:2177)	Edge box sees viewer IP. Subscribe token, stream id scope only.
Enterprise (C)	<code>/assign</code> + connect (main.ts:1582)	Private relay sees viewer IP. Watch + browser-couriered pull token.
Probes	<code>:8888/fingerprint</code> , <code>/edge_xport</code> (main.ts:432,577)	Direct browser→box: box sees browser IP; no token.

Group E — Browser ↔ discovery — REMOVED 2026-08-15

This group no longer exists. It covered the node-id flow: `POST /api/publish` , `POST /api/edge` , and the `relayPut` / `relayGet` calls in `dht.ts` . Every one of those code paths has been deleted — `dht.ts` and `nodeid.ts` are gone from the tree, both Worker routes are gone, and `pkarr` is no longer a dependency. **The browser now contacts no third party outside Cloudflare and the fleet box.**

Closed with it: the un-encrypted node-id media path (there is no longer any path that publishes plaintext — it was removed rather than encrypted), and the world-readable `nodeId → origin EID` record.

Group F — Admin (not a normal-user flow)

Flow	Endpoint	Privacy
Admin	<code>/api/admin/*</code> (index.ts:1511)	⚠ Auth is a hardcoded admin bearer in source (index.ts:1508) — static shared password, a real security weakness (should be a Worker secret). Responses can include email addresses.

4. Chat — the separate plaintext channel

Chat lives in `chat-room.ts` (Durable Object), `chat-client.ts` (browser), route at `index.ts:475` . It is **completely separate from the media path and the relay.**

- **Transport:** WebSocket `wss://<host>/api/streams/<id>/chat` → one **Durable Object per stream**; fan-out to all sockets (chat-room.ts:66).
- **Message on the wire:** client sends `{type:"msg", name, text}` ; DO stamps `{id, name, text, ts}` . **No user id, token, pubkey, stream id, or IP in the message.**

- **Identity = a self-chosen, spoofable display name. No auth required to chat** (route checks only `chat_enabled`). Anyone can chat anonymously. ⚠ **Footgun: if signed in, the client pre-fills the real `user.name` into the handle** (chat-client.ts:22).
- **Storage: the last 50 messages persist in DO storage with no TTL and no deletion on broadcast end** (chat-room.ts:61).
- **Who reads it: plaintext, server-visible** — Cloudflare and the broadcaster see 100% of content + claimed names. The **relay never sees chat**. Not covered by media AES-GCM.
- **IP/metadata:** Cloudflare sees the socket IP at the edge, but the chat code never reads, logs, or stores it; messages carry no IP. Presence = live socket count only.

5. What actually identifies a user

1. **IP + Cloudflare geo/ASN** — ambient on every browser call. The real deanonymizer.
2. **Precise geo (lat/lon)** — captured server-side and **persisted in D1** for broadcasters (`broadcast_events`) and viewers (`watch_events`), and **re-exposed** (one endpoint publicly).
3. **Stream id** — pseudonymous; a stream, not a person.
4. **Chat display name** — self-chosen; self-doxxing vector (esp. auto-filled real name).
5. **Email** — only on the admin surface; inert for normal users (OAuth off).
6. **Tokens carry none of the above.**

6. How a user gets revealed — ranked exposures

HIGH — **Persisted precise geolocation, one endpoint public.** Lat/lon captured from `request.cf` and stored in D1 for **both** broadcasters and viewers, then re-exposed by `GET /api/stats/live` and — critically — the **public** `GET /api/stats/stream/:id/viewers` . Anyone holding a stream id can read the precise stored location of everyone who watched it. Top risk; live today.

HIGH — **Precise viewer geo deliberately forwarded to an external broker.** `brokerHints` sends `{lat,lon,country,colo}` to `moqcdn.net/cdnadmin` on every brokered assign.

MEDIUM — Raw browser IP exposed to third-party infrastructure. Direct browser→relay shows the **untrusted fleet box** the real IP + pseudonymous stream id (network floor). *Partly resolved 2026-08-15*: the pkarr half is gone — no browser contacts a public DHT relay, so no third party outside Cloudflare and the fleet box sees an IP↔content linkage. The network floor against the fleet box remains, and is what a VPN or Tor addresses.

MEDIUM — Chat: plaintext + persistence + name footgun. All chat is plaintext to Cloudflare and the broadcaster; last 50 messages persist with no expiry / no end-of-broadcast deletion; a signed-in user's real name is auto-filled into the handle.

Resolved 2026-08-15 — Node-id media path is not encrypted. The path was *removed* rather than encrypted. There is now exactly one media path and it is always end-to-end encrypted, so "no plaintext" holds everywhere without exception.

LOW / OPERATIONAL — Admin surface. A hardcoded admin bearer (static password in source) guards endpoints that can return email addresses. Fix regardless of the privacy model.

7. What already protects users

- **No accounts.** OAuth off → no durable account identity; every user is anon `id:1`.
- **Clean tokens.** No IP/geo/email/account id in any JWT — only stream id + exp.
- **Relay is content-blind and chat-blind.** Media is ciphertext (fleet flow); chat never touches the relay.
- **No IP in tokens, messages, or chat records.** IP is transient at the edges — except the geo *derived* from it that *is* stored (§6 High).
- **Media path carries no geo;** geo lives only in the control plane.

8. Recommendations (ranked to the high-privacy bar)

1. **Stop persisting precise geo; coarsen to US-state / non-US-country only** (trust-model §7.1) in **both** `broadcast_events` and `watch_events`. Consider **not storing viewer geo at all** — does `watch_events` need to exist?
2. **Lock down or remove the geo-exposing stats endpoints now.** `GET /api/stats/stream/:id/viewers` is public and leaks stored viewer geo; `/api/stats/live`

exposes it broadly. Gate behind the operator or strip geo. (*Live leak — highest urgency.*)

3. **Coarsen the geo forwarded to the broker** (`brokerHints`) to the same label.
4. **Chat:** don't auto-fill the real account name; add a TTL / delete DO storage on broadcast end; document that chat is plaintext-to-Cloudflare (or move to E2E chat later).
5. **Encrypt or retire the node-id media path** so "no plaintext" holds everywhere. **Done 2026-08-15 — retired.**
6. **Replace the hardcoded admin bearer with a Worker secret;** keep emails off any normal-user response.
7. **pkarr IP exposure** (node-id flow): ~~self-host a pkarr relay or proxy put/get through the Worker.~~ **Done 2026-08-15** — resolved by removing the flow, so neither mitigation is needed. Cross-fleet routing never depended on it: the broker already places viewers given the origin.
8. **Relay IP exposure** is the network floor — the user's own tool (Tor/VPN); can't be removed in software. Document honestly.

9. Summary — per flow

IP: peer sees browser IP directly **(Y)** / via Cloudflare **(cf)** / not at all **(N)**. Geo: **precise** = lat/lon, **N** = none.

#	Flow	IP	Geo	Token	Acct	Persists?
A1	go-live	cf	precise → D1	pub JWT	anon	geo → broadcast_events
A2	end	cf	N	N	N	ended_at
B1	route	cf	precise → broker + ASN	viewer/pull + key	N	N
B2	watch start	cf	precise → D1	N	anon	geo → watch_events
B3	live / per-stream viewers	cf	re-exposes stored geo (one PUBLIC)	N	anon	reads D1
C1	broker /cdn/assign	N	precise (hints)	CDN token + pull	N	N
C2	direct assign/release	N	N	provision bearer	N	N
D	browser→relay WT	Y (box)	N	Y (stream-scoped)	N	N
E1 E2	/api/publish, /api/edge, DHT put/get	—	—	—	—	removed 2026-08-15
—	chat WS	cf	N	N	display name	last 50 in DO, no TTL
—	admin	cf	N	static admin bearer	emails	—